DigitalTwin for MyPalletizer 260 M5 Unity Version

[tittle] | *tittle.webp*

Inhaltsverzeichnis

1. Abstract . . . 1
2. Abbildungsverzeichnis . . . 2
3. Introduction and Goals . . . 3
4. Architecture Constraints . . . 5
5. Context and Scope . . . 7
6. Solution Strategy . . . 11
7. Building Block View . . . 13
8. Runtime View . . . 16
9. Deployment View . . . 18
10. Cross-cutting Concepts . . . 20
11. Architecture Decisions . . . 23
12. Quality Requirements . . . 24
13. Risks and Technical Debts . . . 26
14. Glossary . . . 27

1. Abstract

Dieses Dokument beschreibt die Architektur des digitalen Zwillings für den MyPalletizer 260 M5. Das System ermöglicht die hybride Steuerung von Hardware und Simulation über eine einheitliche Python-Schnittstelle. Es dient primär Ausbildungszwecken an der Hochschule Luzern (HSLU).

2. Abbildungsverzeichnis

list::figure[]

3. Introduction and Goals

Das Projekt **DigitalTwin for MyPalletizer 260 M5** zielt darauf ab, eine hochperformante und physikalisch akkurate Simulationsumgebung für den MyPalletizer-Roboterarm zu schaffen. Durch die Bereitstellung einer identischen Programmierschnittstelle für Hardware und Simulation wird der Lernprozess in der Robotik effizienter, sicherer und ortsunabhängiger gestaltet.

3.1. Functional Requirements

Das System erfüllt die folgenden funktionalen Kernanforderungen:

- **Hybride Steuerung**: Nahtlose Umschaltung zwischen `virtual` (Simulation), `real` (Hardware) und `both` (Synchron) Betriebsmodi über eine zentrale Python-Klasse.
- **Gelenksteuerung (Kinematik)**:
  - Präzise Vorgabe von Gelenkwinkeln für alle 4 Achsen des Roboters.
  - Unterstützung von relativen und absoluten Bewegungen.
  - Konfigurierbare Bewegungsgeschwindigkeit zur Anpassung an reale Hardware-Limits.
- **End-Effektor-Simulation**:
  - Steuerung eines parallelen Greifers (Öffnen/Schließen mit variablem Winkel).
  - Simulation einer Vakuumpumpe inklusive physikalischem Ansaugen von Objekten über Trigger-Zonen.
- **Dynamische Szenariengestaltung**:
  - Echtzeit-Erzeugung (Spawn) und Löschung von 3D-Primitiven (Boxen, Kugeln) zur Laufzeit über Python.
  - Zuweisung von physikalischen Eigenschaften (Masse, Reibung) und Farben an gespawnte Objekte.
  - Speichern und Laden von Szenarien-Presets für reproduzierbare Experimente.
- **Visuelles Feedback & Status**:
  - Echtzeit-Darstellung der Roboterbewegung in Unity.
  - Steuerung der am Roboterkopf befindlichen LED-Statusanzeige (RGB-Werte).
  - Verschiedene Kameraperspektiven in Unity zur Analyse komplexer Bewegungsabläufe.

3.2. Quality Goals

Die Architektur orientiert sich an den folgenden Qualitätszielen (angelehnt an ISO 25010):

Ziel	Szenario	Maßnahme
Latenzoptimierung	Ein Bewegungsbefehl muss in < 50ms visuell umgesetzt werden.	Nutzung von UDP (User Datagram Protocol) und Multithreaded Empfangslogik in Unity, um Rendering-Latenzen zu vermeiden.

Ziel	Szenario	Maßnahme
Hardware-Treue	Die Simulation muss physikalische Gelenklimits und Geschwindigkeiten einhalten.	Einsatz von Unity Articulation Bodies mit PID-Regelung und Kalibrierung gegen CAD-Daten.
Wartbarkeit	Neue End-Effektoren (z.B. Kamera-Aufsätze) sollen leicht integrierbar sein.	Modularer Aufbau durch Adapter-Patterns und klare Trennung von Protokoll-Logik und Hardware-Abstraktion.
Portabilität	Die Software muss auf Windows, Linux und macOS identisch funktionieren.	Einsatz von plattformunabhängigen Engines (Python 3.12 / Unity 6).

3.3. Stakeholders

Stakeholder	Rolle	Interessen / Erwartungen
Studierende (HSLU)	Endnutzer	Einfache Installation ("Zero-Config"), intuitive Python-API, fehlerverzeihende Simulationsumgebung (kein Hardware-Schaden bei Fehlern).
Dozierende (APPLAI Modul)	Auftraggeber / Prüfer	Verlässliches Tool für die Lehre, einfache Korrektur von Studentenprojekten ohne Hardware-Zwang, hohe Reproduzierbarkeit von Ergebnissen.
Robotik-Enthusiasten	Externe Nutzer	Open-Source Verfügbarkeit, gute Dokumentation zur Erweiterung auf eigene Szenarien.
Entwickler (Colin Felber)	Architekt	Klare Strukturierung zur Minimierung von technischer Schuld, Dokumentation als Leistungsnachweis für das Studium.

4. Architecture Constraints

Die Architektur des Digital Twins unterliegt technischen, organisatorischen und konventionellen Rahmenbedingungen, die den Lösungsraum definieren.

4.1. Technische Rahmenbedingungen

Randbedingung	Erläuterung
Programmiersprache	Die Steuerungsschicht muss in **Python (v3.12+)** implementiert sein, um die Kompatibilität mit bestehenden Lernressourcen und der `pymycobot`-Library sicherzustellen.
Simulations-Engine	Als Visualisierungs-Engine wird **Unity** verwendet. Dies erfordert eine C#-basierte Implementierung der Empfängerlogik und die Nutzung von Unity-spezifischen Komponenten wie `ArticulationBody`.
Kommunikation	Die Ansteuerung der physischen Hardware erfolgt über eine **serielle Schnittstelle (UART via USB)** mit einer Standard-Baudrate von 115200.
Inter-Prozess-Kommunikation	Da Steuerlogik (Python) und Simulation (Unity) in getrennten Prozessen laufen, muss ein Netzwerkprotokoll (**UDP**) zur Datenübertragung genutzt werden.
Hardware-Plattform	Das System muss auf Standard-Laptops (Windows 10/11, Ubuntu 22.04+, macOS) lauffähig sein, ohne dass dedizierte Grafikkarten zwingend erforderlich sind.

4.2. Organisatorische Rahmenbedingungen

- **Open Source & Reproduzierbarkeit**: Das Projekt wird auf GitHub verwaltet. Alle Abhängigkeiten (wie `pymycobot`, `numpy`) müssen über Standard-Paketmanager (`pip`) installierbar sein.
- **Zeitrahmen**: Die Entwicklung erfolgt iterativ innerhalb eines Semesters im Modul APPLAI.
- **Lizenzierung**: Es werden nur Bibliotheken und Assets verwendet, die für akademische Zwecke frei verfügbar sind.

4.3. Konventionen und Standards

- **ARC42**: Die Dokumentation folgt strikt dem ARC42-Standard.
- **C4-Modell**: Für Architekturdiagramme werden die Prinzipien des C4-Modells (Context, Container, Components) angewendet.
- **Coding Standards**:
  - Python: Einhaltung von PEP 8.
  - C#: Unity Coding Conventions (PascalCase für öffentliche Member, SerializedFields für

Editor-Zugriff).

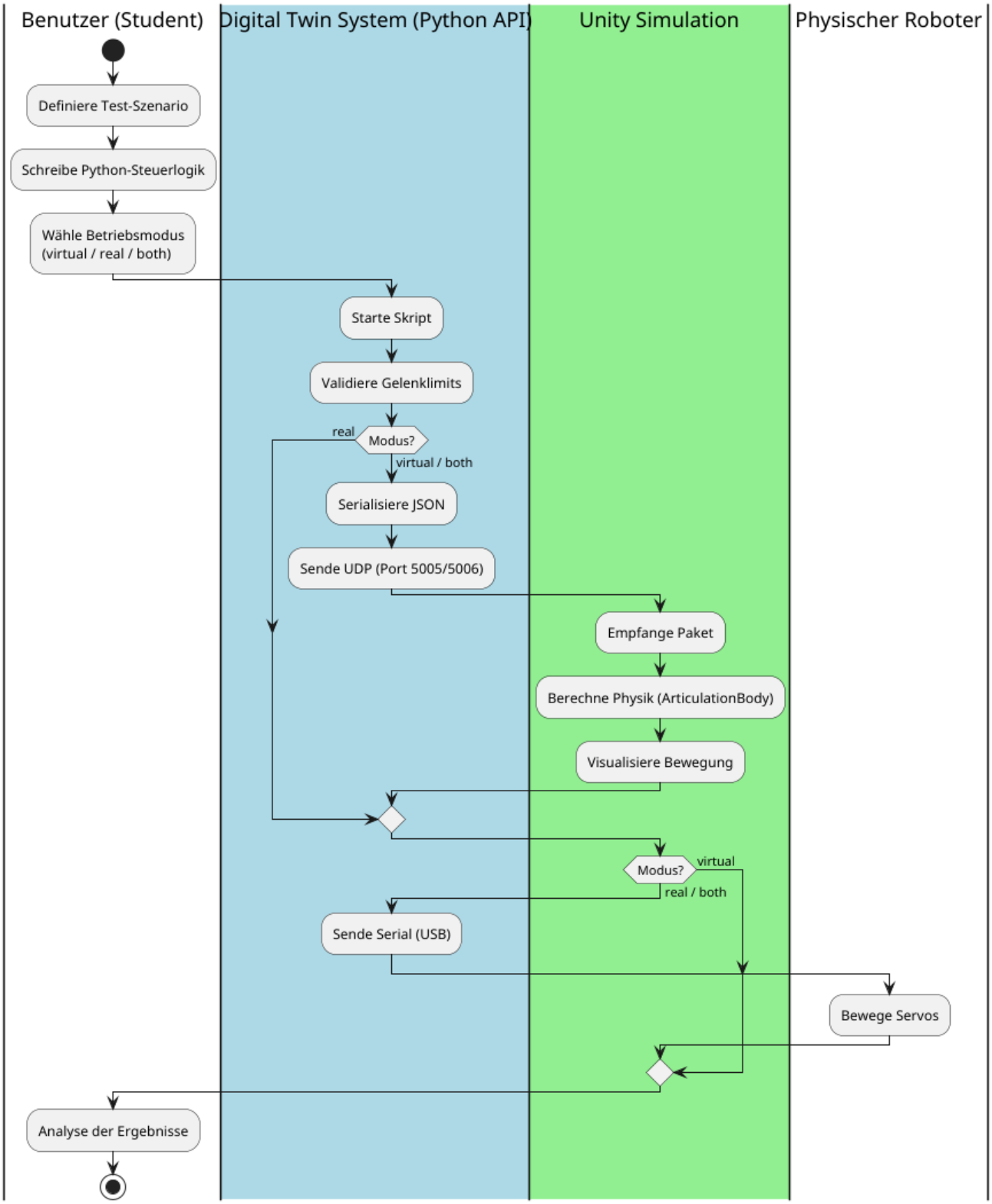
- Kommunikation: Versionierung des JSON-Protokolls (aktuell v1).

5. Context and Scope

In diesem Kapitel wird die Abgrenzung des Systems gegenüber externen Akteuren definiert sowie der operative Prozess (Workflow) dargestellt.

5.1. Business Context

Der Business Context beschreibt den Prozess aus Sicht der Anwender (Studierende/Dozierende). Das folgende Diagramm im BPMN-Stil visualisiert den typischen Ablauf einer Simulations-Session.

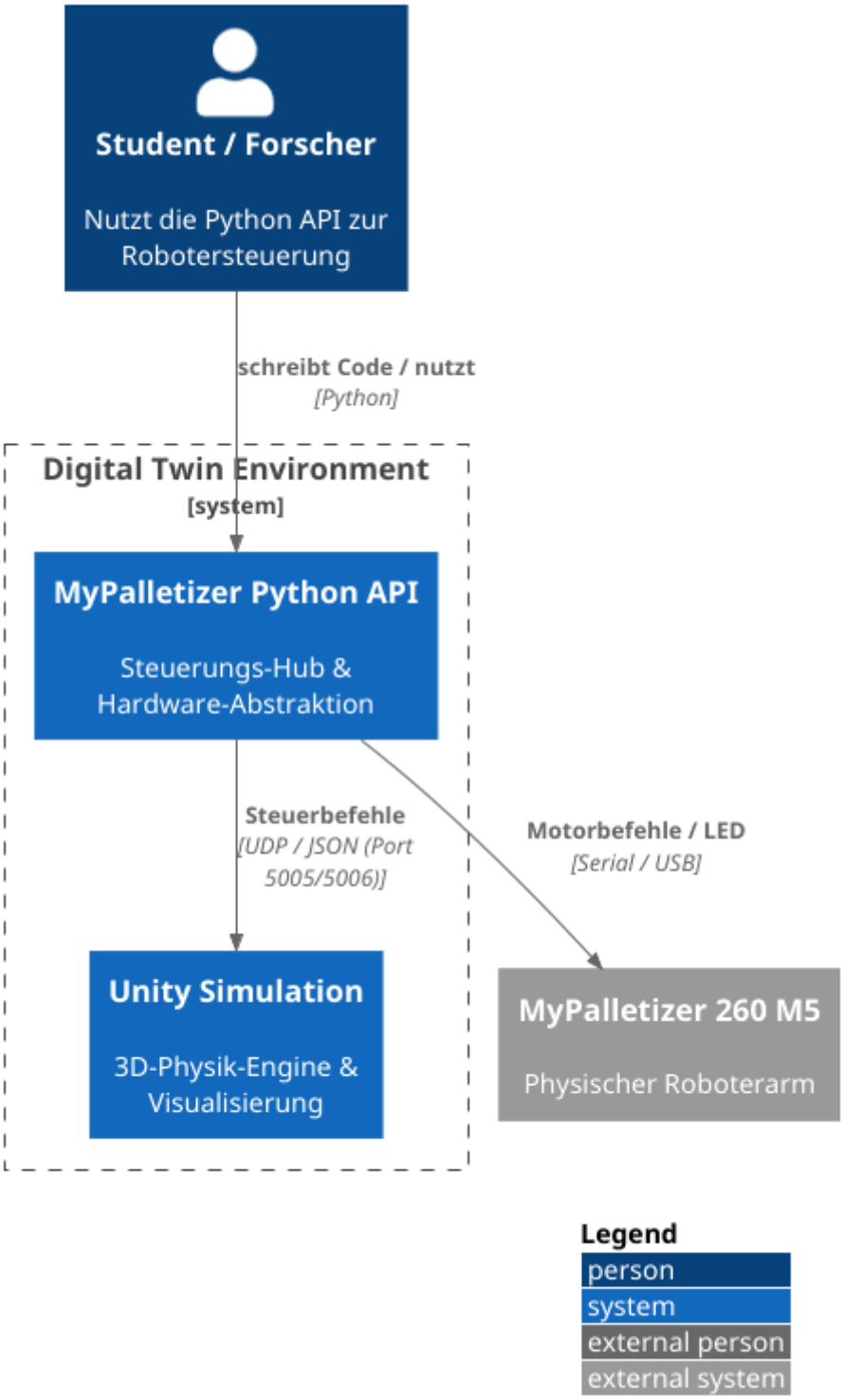


Akteur	Verantwortung im Prozess
Benutzer	Erstellt die Programmlogik und bewertet das Verhalten des Roboters in der Simulation oder an der Hardware.
Python API	Übernimmt die Orchestrierung, die Validierung der Parameter und die Kommunikation über die verschiedenen Protokolle.
Unity Simulation	Bietet die visuelle und physikalische Validierung der geplanten Bewegungsabläufe.

Akteur	Verantwortung im Prozess
Hardware	Führt die validierten Befehle in der realen Welt aus.

5.2. Technical Context

Der technische Kontext zeigt die Systemgrenzen und die spezifischen Kommunikationskanäle zwischen den Komponenten.



5.2.1. Externe Schnittstellen

Schnittstelle	Protokoll	Zweck
Benutzer-API	Lokal (Python)	High-Level Funktionen wie `send_angles()`, `set_color()` oder `spawn_cube()`.
Hardware-Kanal	Serial (115200 Baud)	Übertragung von Low-Level Befehlen an die M5Stack-Firmware des Roboters.
Simulations-Kanal	UDP (v1 JSON)	Hochfrequente Übertragung von Soll-Zuständen an die Unity-Engine auf Port 5005.
Welt-Kanal	UDP (v1 JSON)	Übertragung von Umgebungsbefehlen auf Port 5006.

6. Solution Strategy

Die Lösungsstrategie basiert auf einer strikten Trennung zwischen der logischen Steuerungsebene (Python) und der physikalischen Darstellungsebene (Unity).

6.1. Architekturelle Leitprinzipien

1. **Einheitliche Abstraktion**: Der Benutzer soll nicht wissen müssen, ob er gerade eine Simulation oder Hardware steuert. Alle modusspezifischen Unterschiede werden innerhalb des `MyPalletizerController` gekapselt.
2. **Entkopplung durch Netzwerkschnittstellen**: Die Nutzung von UDP ermöglicht es, Steuerlogik und Visualisierung auf verschiedenen Rechnern oder in verschiedenen Prozessen laufen zu lassen. Dies schützt den Python-Prozess vor Performance-Schwankungen der 3D-Engine.
3. **Zustandslosigkeit des Protokolls**: Jedes UDP-Paket enthält den vollständigen Soll-Zustand (z.B. alle 4 Gelenkwinkel). Dies macht die Kommunikation robust gegenüber Paketverlusten, da das nächste Paket den Zustand automatisch korrigiert.

6.2. Entscheidungen zur Kommunikation

- **Warum UDP statt TCP?** In einer Echtzeit-Simulation ist Aktualität wichtiger als Vollständigkeit. Ein verloren gegangenes Paket ist unkritisch, da Millisekunden später ein neues Paket mit dem aktuellen Zustand folgt. TCP-Retransmissions würden hier zu unerwünschten "Sprüngen" und Latenzen führen.
- **Warum JSON?** JSON bietet eine hervorragende Balance zwischen Lesbarkeit (Debugging) und Performance. Da die Datenmenge pro Paket (ca. 200 Bytes) sehr gering ist, überwiegen die Vorteile der einfachen Implementierung in C# und Python gegenüber binären Formaten wie Protobuf.

6.3. Schichtenmodell

Schicht	Verantwortung
User Logic	Das Python-Skript des Studenten. Enthält die Pfadplanung oder KI-Logik.
Controller Layer	Die `mypalletizer`-Library. Handhabt die Verteilung auf Serial/UDP und validiert Gelenklimits.
Transport Layer	JSON Serialisierung und UDP/Serial Dispatching.
Execution Layer	Unity Articulation Bodies (Simulation) oder Robot Firmware (Hardware).

6.4. Strategie zur Welt-Manipulation

Die Welt-Manipulation (Spawning von Objekten) wurde bewusst auf einen separaten Port (5006) ausgelagert. Während Roboterbewegungen als kontinuierlicher Stream (Stream-orientiert)

betrachtet werden, sind Welt-Anderungen diskrete Ereignisse (Event-orientiert). Diese Trennung verhindert, dass ein großes JSON-Paket für eine Welt-Änderung den Fluss der Bewegungsdaten unterbricht.

7. Building Block View

Die Bausteinsicht beschreibt die statische Struktur des Systems durch eine hierarchische Zerlegung in funktionale Komponenten.

7.1. Whitebox Overall System (Level 1)

Auf der obersten Ebene besteht das System aus zwei Hauptcontainern: der Python Control Layer und der Unity Visualization Layer.

7.1.1. Contained Building Blocks

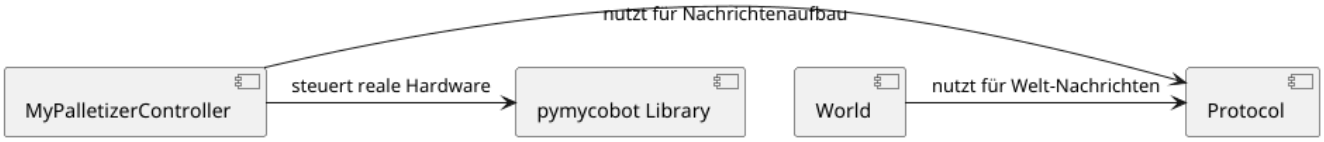
Baustein	Beschreibung
Python Control Layer	Diese Schicht bildet das "Gehirn" des Systems. Sie stellt die Schnittstelle zum Benutzer bereit und entscheidet über die Weiterleitung der Befehle an Hardware oder Simulation.

Baustein	Beschreibung
Unity Visualization Layer	Diese Schicht fungiert als "Körper". Sie empfängt Datenpakete und setzt diese in physikalische Bewegungen oder visuelle Änderungen in der 3D-Szene um.
Schnittstelle: UDP/JSON	Ein asynchrones Nachrichtenprotokoll zur entkoppelten Kommunikation zwischen beiden Schichten.

7.2. Level 2

In dieser Ebene werden die internen Strukturen der beiden Hauptcontainer detailliert.

7.2.1. Whitebox Python Control Layer



- **MyPalletizerController**: Zentrale Klasse zur Orchestrierung. Implementiert das Strategie-Pattern für die Betriebsmodi.
- **World**: Verwalter der Simulationsumgebung. Kapselt Befehle zum Erzeugen und Löschen von Objekten.
- **Protocol**: Hilfsmodul zur zustandslosen Serialisierung von Befehlen in das JSON-Format (v1).

7.2.2. Whitebox Unity Visualization Layer

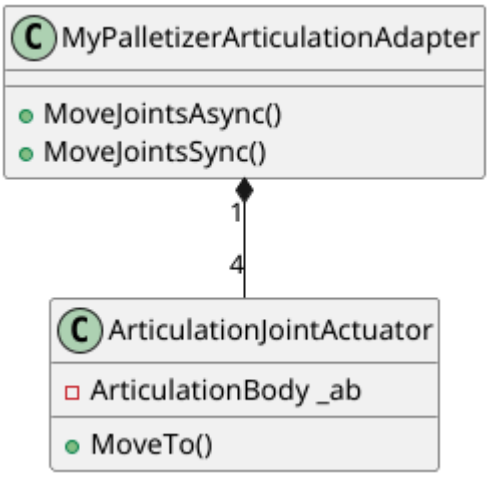
- **UnityUdpBridge**: Multithreaded Empfangseinheit für Roboter-relevante Daten auf Port 5005.
- **WorldUdpBridge**: Empfangseinheit für Welt-Manipulationen auf Port 5006.
- **MyPalletizerAdapter**: High-Level Controller, der die physikalische Gelenkkette koordiniert.
- **WorldSpawnManager**: Verarbeitet die Instanziierung von GameObjects basierend auf den empfangenen JSON-Daten.

7.3. Level 3

Auf Level 3 betrachten wir die feinmodularen Komponenten innerhalb der Unity-Simulation, die für die physikalische Präzision verantwortlich sind.

7.3.1. Whitebox MyPalletizerAdapter & Gelenke

Innerhalb des `MyPalletizerAdapter` werden vier `ArticulationJointActuator`-Komponenten gesteuert, die jeweils ein physikalisches Gelenk repräsentieren.



- **ArticulationJointActuator**: Diese Komponente ist das Herzstück der Bewegungssimulation. Sie greift direkt auf Unitys `ArticulationBody` zu und nutzt Coroutinen, um eine geschwindigkeitsbasierte Bewegung (`degPerSec`) zu simulieren. Dies stellt sicher, dass der Roboterarm nicht "teleportiert", sondern physikalisch plausibel schwenkt.
- **ArticulationBody (Unity-intern)**: Verwendet den **Reduced Coordinate Space** zur stabilen Simulation der kinematischen Kette ohne Drifts.

8. Runtime View

Die Laufzeitsicht verdeutlicht das Zusammenspiel der Komponenten über Prozessgrenzen hinweg. Ein kritischer Aspekt ist dabei das Threading-Modell in Unity.

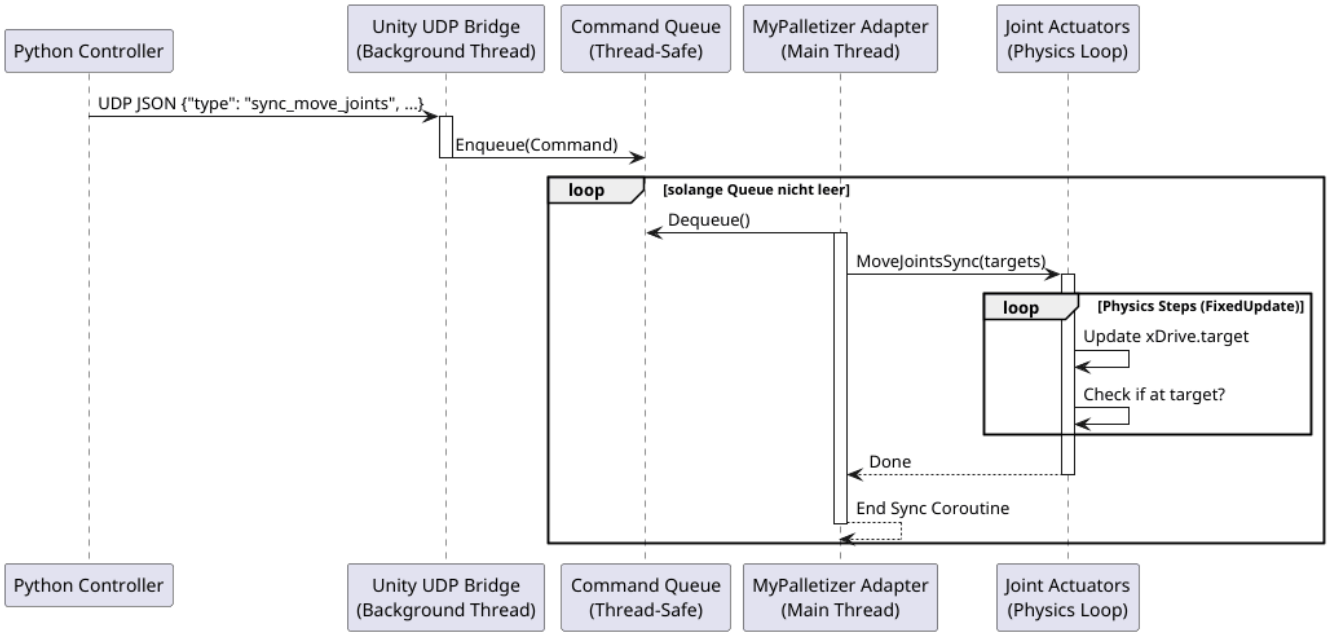
8.1. Szenario 1: Asynchrone Gelenkbewegung (Fire-and-Forget)

Dies ist der Standard-Ablauf für flüssige Bewegungen. Der Python-Thread sendet Datenpakete in hoher Frequenz, ohne auf eine Antwort zu warten.

1. **Python Side**: `send_angles(j1, j2, j3, j4, speed)`
    - Die Werte werden serialisiert und als UDP-Datagramm verschickt.
2. **Unity Network Thread**: `UdpClient.Receive()`
    - Der Thread blockiert, bis ein Paket eintrifft.
    - Das JSON wird geparst und ein `RobotCommand`-Objekt in die `ConcurrentQueue` eingereiht.
3. **Unity Main Thread**: `Update()` / `CommandRunner()`
    - Im nächsten Frame wird der Befehl aus der Queue entnommen.
    - Der `ArticulationAdapter` startet eine Coroutine für jedes Gelenk via `ArticulationJointActuator`.
    - Die Physik-Engine berechnet in jedem `FixedUpdate` die neue Position basierend auf `degPerSec`.

8.2. Szenario 2: Synchrone Gelenkbewegung (Sync Move)

Obwohl UDP verbindungslos ist, bietet die API einen `sync_move_joints`-Befehl an. In der Hardware-Welt wartet der Roboter, bis die Gelenke die Zielposition erreicht haben. In der Simulation wird dies durch Coroutinen nachgebildet.



8.3. Multithreading-Konzept in Unity

Ein wesentliches Merkmal der Implementierung ist die Lösung des "Main-Thread-Only"-Problems von Unity. Da Netzwerknachrichten jederzeit eintreffen können, Unity-APIs (wie `ArticulationBody`) aber nur im Haupt-Thread aufgerufen werden dürfen, fungiert die `ConcurrentQueue` als Brücke.

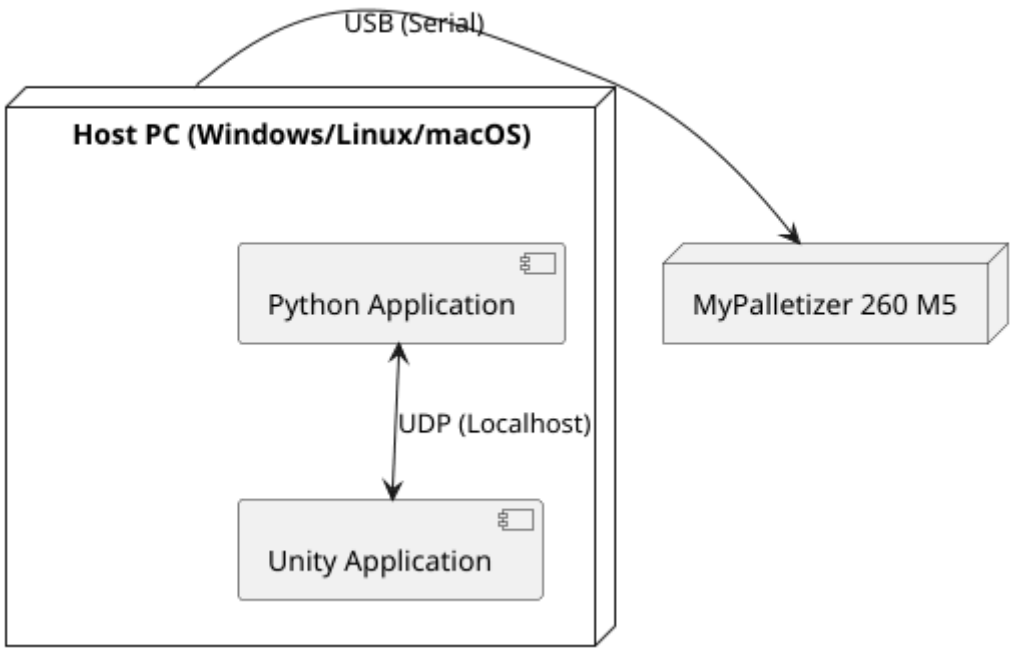
- **Performance-Vorteil**: Das Rendering der Simulation ruckelt nicht, selbst wenn das Netzwerk kurzzeitig hohe Latenzen aufweist oder viele Pakete gleichzeitig eintreffen.
- **Datensicherheit**: Die Nutzung einer thread-sicheren Queue verhindert Race Conditions zwischen dem Netzwerk-Thread und der Physik-Berechnung.

9. Deployment View

Die Verteilungssicht beschreibt die physische Infrastruktur und die Softwareumgebung, in der das System betrieben wird.

9.1. Infrastructure Level 1

Das System ist für den Betrieb auf einer Standard-Workstation ausgelegt. Die optionale Hardware wird lokal per USB angebunden.



9.2. Infrastructure Level 2

Auf Ebene 2 betrachten wir die spezifischen Anforderungen und Komponenten der Betriebsumgebung.

9.2.1. Host-PC (Workstation)

Der Host-PC benötigt folgende installierte Softwarekomponenten:

- **Python Runtime (v3.12)**:
  - Erforderlich für die Ausführung der Steuerlogik.
  - Beinhaltet Abhängigkeiten wie `pymycobot`, `numpy` und `pyserial`.
- **Unity-Standalone-Build**:
  - Die vorkompilierte Version der Simulation. Diese enthält alle Assets und die C#-Receiver-Logik.
- **USB-Treiber (CP210x)**:
  - Erforderlich für die Kommunikation mit dem M5Stack-Controller des Roboters über den

virtuellen COM-Port.

9.2.2. Netzwerk-Infrastruktur

Obwohl das System primär auf `localhost` (127.0.0.1) betrieben wird, ermöglicht die UDP-basierte Architektur eine Verteilung der Komponenten im lokalen Netzwerk (LAN/WLAN):

Element	Beschreibung
Port 5005 (Roboter)	Eingehender UDP-Verkehr für Bewegungsdaten. Muss in der Firewall des Host-PCs freigegeben sein.
Port 5006 (Welt)	Eingehender UDP-Verkehr für Umgebungsbefehle.
Loopback Adapter	Minimale Latenz (< 1ms) bei lokalem Betrieb.

9.2.3. MyPalletizer 260 M5 (Hardware)

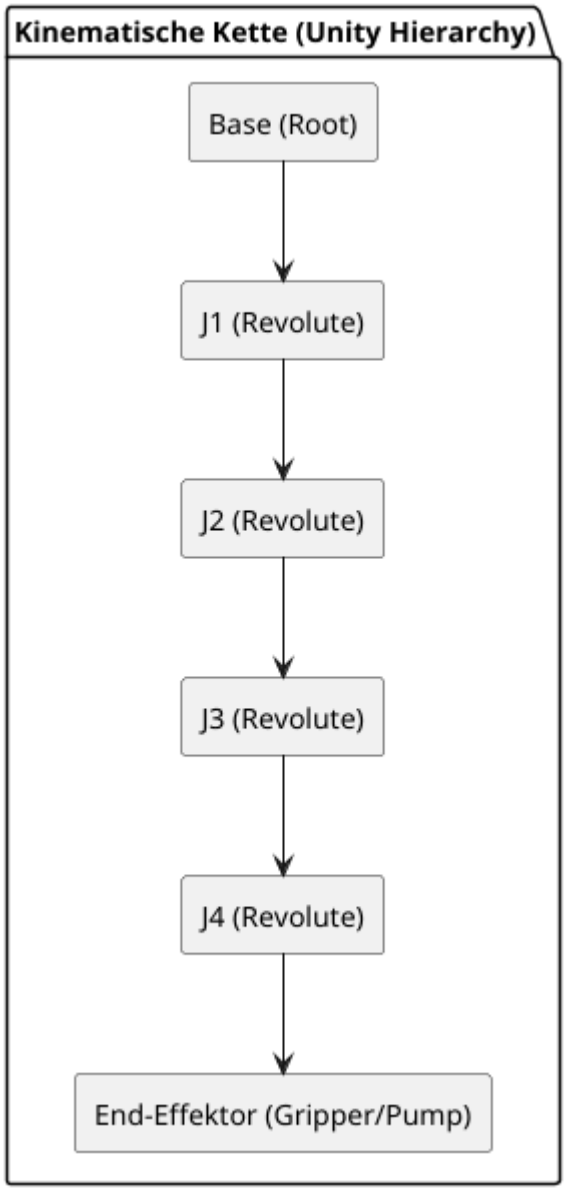
- **Firmware**: Auf dem Roboter muss die aktuelle Version von **atomMain** oder **Transponder** installiert sein, um die seriellen Befehle korrekt zu verarbeiten.
- **Energieversorgung**: Ein externes Netzteil (8.4V / 5A) ist für den Betrieb der Servos zwingend erforderlich, da die Stromversorgung über USB nicht ausreicht.

10. Cross-cutting Concepts

In diesem Kapitel werden die technischen Konzepte erläutert, die systemübergreifend implementiert wurden.

10.1. Physik-Konzept: Articulation Bodies

Die Simulation des MyPalletizer nutzt Unitys `ArticulationBody`-Komponente anstelle von Standard-Rigidbodies.

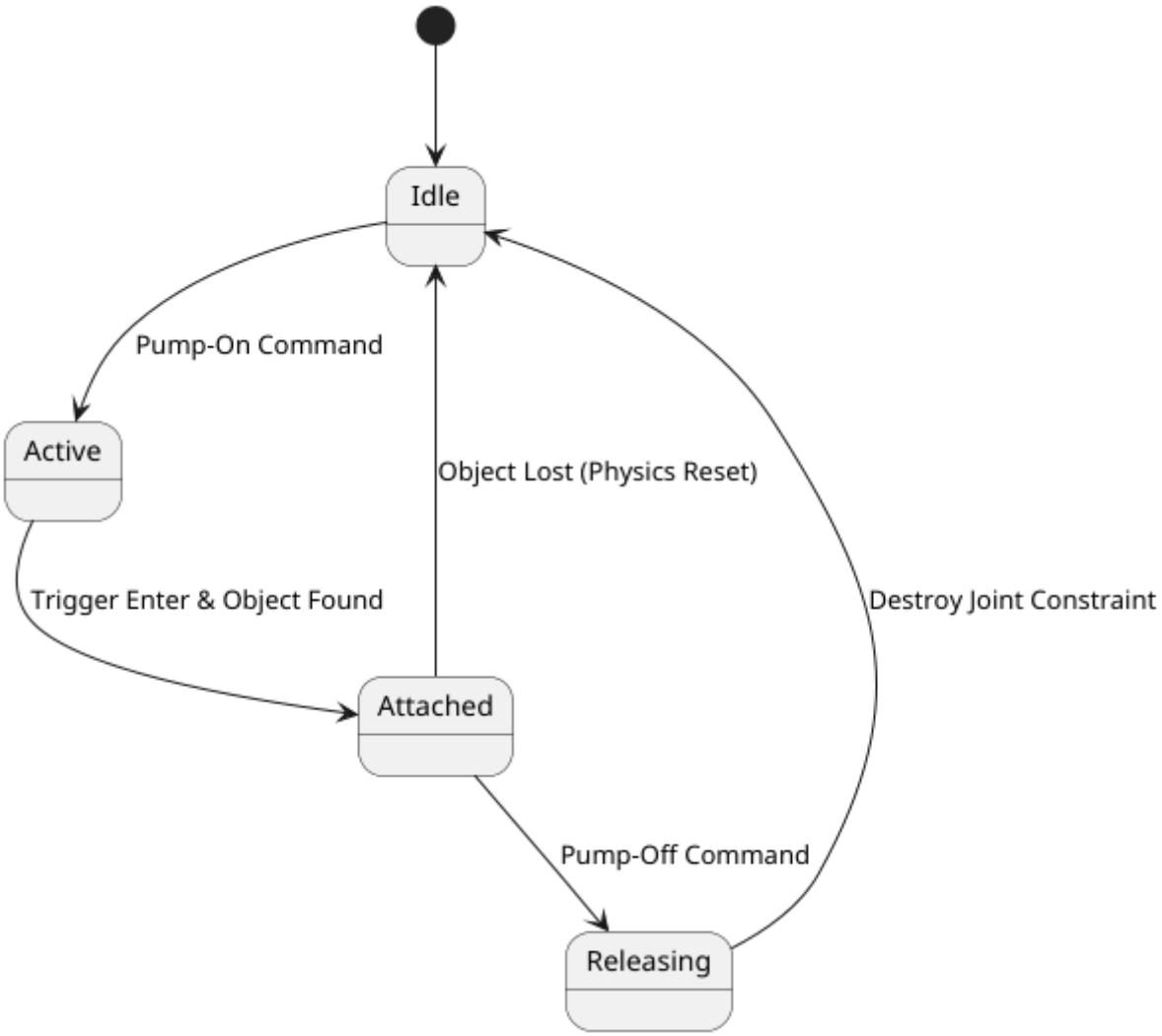


- **Vorteil**: Articulation Bodies verwenden "Reduced Coordinates" (Gelenkkoordinaten statt kartesischer Koordinaten). Dies verhindert das Auseinanderdriften der Gelenke bei schnellen Bewegungen oder Kollisionen, was bei Rigidbodies mit Joints oft ein Problem darstellt.
- **Steuerung (Drive Mode)**: Die Gelenke werden im `Targets`-Modus betrieben. Jedes Gelenk hat einen PID-Regler (Proportional-Integral-Derivative), der über die Parameter `Stiffness` (P) und `Damping` (D) konfiguriert wird.
- **Gelenk-Mapping**:

- `j1`: Revolute (Basis-Rotation)
- `j2`: Revolute (Unterarm)
- `j3`: Revolute (Oberarm)
- `j4`: Revolute (Handgelenk) Jeder `ArticulationJointActuator` übersetzt die eingehenden Winkel in den `target`-Wert des jeweiligen Drives.

10.2. End-Effektor-Konzept

Der MyPalletizer unterstützt verschiedene Werkzeuge, die in der Simulation funktional nachgebildet wurden:



- **Greifer (Gripper)**: Wird über zwei gegenläufige `ArticulationBodies` simuliert. Ein `GripperController` übersetzt den binären Status (`open`/`close`) in Zielwinkel für die Greiferfinger. Physikalische Materialien mit hoher Reibung ermöglichen das Greifen von Objekten.
- **Saugpumpe (Suction Pump)**: Dieses Werkzeug nutzt ein physikalisches Constraint (z.B. ein `ConfigurableJoint`), das dynamisch erzeugt wird, wenn die Pumpe aktiviert ist und ein `GrabbableObject` den Trigger-Bereich berührt. Dies simuliert das "Ansaugen" realitätsnah.

10.3. Welt-Synchronisation & Objekt-Management

Um komplexe Szenarien reproduzierbar zu machen, implementiert das System ein ID-basiertes Objekt-Management.

- **Dynamisches Spawning**: Objekte erhalten beim Erzeugen in Python einen eindeutigen Namen. Dieser Name dient in Unity als Schlüssel, um das Objekt später gezielt zu verschieben oder zu löschen.
- **Prefab-System**: Unity hält eine Bibliothek von Prefabs (Box, Kugel, Zylinder) bereit. Der Python-Befehl definiert lediglich den Typ, die Position und die Skalierung.

10.4. Test-Konzept

Die Qualitätssicherung erfolgt auf drei Ebenen:

1. **Unit Tests (Python)**: Validierung der `protocol.py`. Testet, ob z.B. negative Geschwindigkeiten korrekt abgefangen oder auf den erlaubten Bereich begrenzt werden.
2. **Integration Tests (UDP)**: Ein automatisiertes Skript sendet Test-Patterns an Unity und prüft (via Log-Analyse), ob alle Pakete in der richtigen Reihenfolge verarbeitet wurden.
3. **Visuelle Validierung (Physik)**: Manuelle Tests zur Überprüfung der Kollisionserkennung. Beispiel: Kann der Roboter eine Box von A nach B heben, ohne dass diese durch den Greifer "rutscht"?

11. Architecture Decisions

Die folgenden Architectural Decision Records (ADR) dokumentieren die wegweisenden Entwurfsentscheidungen des Projekts.

11.1. ADR 1: Nutzung von Unity Articulation Bodies

- **Kontext**: Die Standard-Physik von Unity (Rigidbodies + Joints) neigt bei Roboterarmen mit vielen Gelenken zu Instabilitäten ("Jittering").
- **Entscheidung**: Einsatz der `ArticulationBody`-Komponente, die speziell für Roboterkinematik (Reduced Coordinates) entwickelt wurde.
- **Konsequenz**: Höhere Stabilität der Gelenkkette, aber auch komplexere Konfiguration der PID-Werte (`Stiffness`, `Damping`).

11.2. ADR 2: UDP-Kommunikation für Visualisierung

- **Kontext**: Die Visualisierung muss synchron zur Hardware erfolgen (Latenz kritisch).
- **Entscheidung**: Wahl von UDP gegenüber TCP oder WebSockets.
- **Konsequenz**: Minimale Latenz. Paketverluste werden durch die hohe Sendefrequenz und die Übertragung des absoluten Zustands (statt Differenzwerten) kompensiert.

11.3. ADR 3: Zwei-Kanal-Trennung (Control vs. World)

- **Kontext**: Bewegungsdaten sind hochfrequent, Welt-Manipulationen sind sporadisch, aber datenintensiv.
- **Entscheidung**: Trennung auf Port 5005 (Control) und Port 5006 (World).
- **Konsequenz**: Ein großes JSON-Paket für die Welt-Manipulation blockiert nicht den asynchronen Empfang der Gelenkwinkel.

11.4. ADR 4: Threaded UDP-Receiver in Unity

- **Kontext**: `UdpClient.Receive()` blockiert den ausführenden Thread. Ein Aufruf im Main-Thread von Unity würde das Rendering stoppen, bis ein Paket eintrifft.
- **Entscheidung**: Auslagerung des Receivers in einen dedizierten C#-Hintergrund-Thread.
- **Konsequenz**: Flüssiges Rendering der Simulation unabhängig vom Netzwerkverkehr. Erfordert thread-sichere Übergabe an den Main-Thread (ConcurrentQueue).

12. Quality Requirements

Die Qualität des Systems wird durch spezifische Szenarien messbar gemacht, die auf den in Kapitel 1 definierten Zielen basieren.

12.1. Quality Requirements Overview

Das System priorisiert **Performance** (Echtzeit-Feedback) und **Usability** (Einfachheit der Nutzung für Studenten). Da es sich um ein Bildungstool handelt, ist zudem die **Fehlertoleranz** der Simulation von hoher Bedeutung.

Qualitätsmerkmal	Priorität
Performance (Latenz)	Sehr Hoch
Portabilität	Hoch
Wartbarkeit	Mittel
Zuverlässigkeit	Hoch

12.2. Quality Scenarios

ID	Qualitätsziel	Szenario
QS-1	Performance	Ein Benutzer ruft `send_angles` auf. Der Roboterarm in Unity muss innerhalb von spätestens 50ms beginnen, sich zur Zielposition zu bewegen. Gemessen wird die Zeit von der UDP-Paketerzeugung bis zum Start der Coroutine in Unity.
QS-2	Portabilität	Ein Student wechselt von einem Windows-Laptop zu einem MacBook. Das gleiche Python-Skript muss nach Installation der Abhängigkeiten (`pip install`) ohne Code-Änderung lauffähig sein und die Simulation steuern.
QS-3	Robustheit	Die Unity-Simulation wird während der Ausführung eines Python-Skripts hart beendet. Das Python-Skript darf dadurch nicht blockieren oder abstürzen (UDP-Timeout-Freiheit).
QS-4	Physikalische Treue	Ein Gelenkbefehl über den maximalen Winkel hinaus wird gesendet. Der `MyPalletizerController` muss den Wert auf das physikalische Limit klammern (Clamp), bevor er an Hardware oder Simulation gesendet wird.

ID	Qualitätsziel	Szenario
QS-5	Bedienbarkeit	Ein neuer Benutzer installiert die Software. Innerhalb von 15 Minuten soll er in der Lage sein, ein Demo-Skript auszuführen und eine Bewegung in der Simulation zu sehen.

13. Risks and Technical Debts

Dieses Kapitel identifiziert potenzielle Schwachstellen und technische Kompromisse, die während der Entwicklung eingegangen wurden.

13.1. Architektur-Risiken

Risiko	Erläuterung & Minderung
Netzwerk-Instabilität (UDP)	Da UDP keine Zustellung garantiert, könnten Pakete verloren gehen. **Minderung**: Das Protokoll ist zustandslos und sendet absolute Winkel. Das nächste Paket korrigiert automatisch einen verloren gegangenen Zustand.
Synchronitäts-Drift	Im Modus `both` könnte die Hardware langsamer reagieren als die Simulation (oder umgekehrt). **Minderung**: Nutzung identischer `speed`-Parameter in beiden Systemen. Langfristig wäre ein Rückkanal für Ist-Werte (Feedback Loop) notwendig.
Port-Konflikte	Die Standard-Ports 5005/5006 könnten belegt sein. **Minderung**: Die Ports sind im `MyPalletizerController` (Python) und in der `UnityUdpBridge` konfigurierbar.

13.2. Implementierungs-Risiken & Technische Schulden

- **Abhängigkeit von Unity Articulation Bodies**: Die Stabilität der Simulation hängt stark von der Unity-Version (empfohlen 2021.3 LTS oder neuer) ab. Ein Downgrade würde die physikalische Konsistenz gefährden.
- **Open-Loop Steuerung**: Die Python-API setzt voraus, dass Befehle erfolgreich ausgeführt werden. Es findet keine Prüfung statt, ob eine physische oder virtuelle Kollision den Roboter gestoppt hat. Dies ist eine bewusste Entscheidung zur Reduktion der Komplexität.
- **Modell-Präzision**: Die physikalischen Parameter wie Reibung und Trägheit sind Näherungswerte. Für hochpräzise Anwendungen müsste eine Systemidentifikation am realen Roboter durchgeführt werden.
- **API-Abhängigkeit (`pymycobot`)**: Änderungen in der proprietären Herstellerbibliothek können direkte Auswirkungen auf den `real`-Modus haben. Eine Entkopplung durch einen eigenen Hardware-Abstraktions-Layer in Python wäre eine sinnvolle zukünftige Erweiterung.

14. Glossary

Term	Definition
Digital Twin	*Ein digitaler Zwilling ist ein virtuelles Abbild eines physischen Objekts (hier: MyPalletizer 260 M5), das dessen Verhalten und Zustand in Echtzeit simuliert.*
Articulation Body	*Eine spezielle Komponente in Unity zur physikalischen Simulation von kinematischen Ketten (Robotern), die für hohe Stabilität bei Gelenkbewegungen sorgt.*
UDP	*User Datagram Protocol. Ein verbindungsloses Netzwerkprotokoll, das für seine geringe Latenz bekannt ist.*
JSON	*JavaScript Object Notation. Ein kompaktes Datenformat zum Austausch von strukturierten Daten zwischen Python und Unity.*
pymycobot	*Die offizielle Python-Library des Herstellers Elephant Robotics zur Ansteuerung der MyRobot-Serie.*
APPLAI	*Applied Artificial Intelligence: Das Modul an der HSLU, in dem dieses Projekt entwickelt wurde.*
HSLU	*Hochschule Luzern*

HSLU Hochschule Luzern